

Lenguajes para Análisis de Datos II

Modulo 1

Objetivo

La ciencia de datos es una disciplina que se ocupa de la extracción de conocimiento a partir de los datos. Para ello, los científicos de datos utilizan una variedad de herramientas y técnicas para analizarlos, identificar patrones y tendencias, y tomar decisiones informadas.

Python es un lenguaje de programación de propósito general que es muy popular entre los científicos de datos. Es fácil de aprender, tiene una gran comunidad de usuarios y una amplia gama de bibliotecas y herramientas disponibles.



Entorno de desarrollo integrado (IDE)

Un IDE o entorno de desarrollo integrado es una herramienta que facilita la escritura, edición y ejecución de código, mejorando la experiencia de programación por las diversas funcionalidades que proporciona. Algunas de las características que tienen en común estas herramientas son:

- **Editor de código:** Permite escribir y editar código de manera cómoda, con resaltado de sintaxis, autocompletado de código y otras funciones útiles para el programador.
- **Compilador o intérprete:** Ejecuta el código escrito y muestra los resultados o errores.
- **Depurador:** Permite identificar y corregir errores en el código.
- **Herramientas de administración de proyectos:** Facilita la organización y gestión de proyectos de programación complejos.
- **Integración con herramientas de control de versiones:** Permite realizar un seguimiento de los cambios realizados en el código y colaborar con otros desarrolladores



¿Por qué usar Anaconda y Spyder?

La lista de IDEs que se pueden emplear con Python es amplia, por lo que elegir una puede resultar complicado. En este caso, emplearemos Spyder, que viene instalado en Anaconda, una distribución de Python.

Anaconda cuenta con una gran cantidad de bibliotecas y herramientas populares para la ciencia de datos, el análisis de datos y el machine learning. Por su parte, Spyder ofrece algunas características que lo hacen ideal para trabajar con Python dentro del ámbito de la ciencia de datos, como por ejemplo:

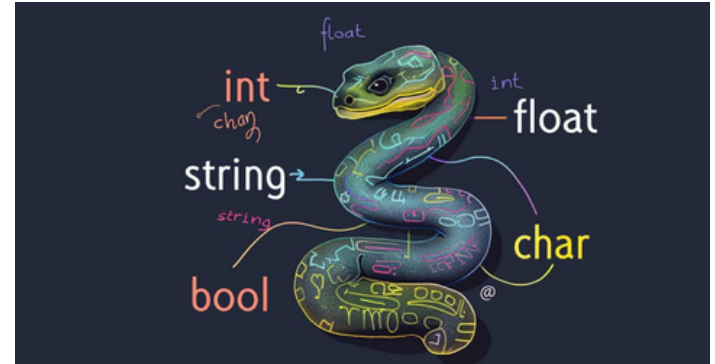
- **Consola integrada:** Permite ejecutar código Python directamente desde el IDE.
- **Explorador de variables:** Muestra las variables y sus valores durante la ejecución del código.
- **Herramientas de visualización de datos:** Permite crear gráficos y visualizaciones de manera sencilla.
- **Integración con IPython:** Facilita el uso de las potentes funcionalidades de IPython, un shell interactivo que añade algunas características extra al modo interactivo que viene incluido con Python de base, algo muy útil para el análisis de datos.



ANACONDA®

Las variables en Python

Las variables son uno de los bloques de creación fundamentales de los programas escritos en Python. Son como contenedores que almacenan datos en la memoria de la computadora. Cada variable tiene un nombre único que se utiliza para referirse a su valor almacenado. Los tipos de datos en Python representan el tipo de información que una variable puede contener, como números, cadenas de texto, listas, etc.



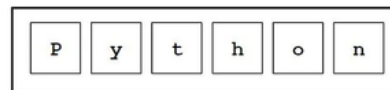
Tipos de variables

Cadenas de textos (str): Representan secuencias de caracteres. Las cadenas de texto se pueden encerrar entre comillas simples (') o dobles ("), y permiten almacenar y manipular texto en programa

Booleanos (bool): Representan dos posibles valores: True (verdadero) y False (falso). Los valores booleanos son fundamentales para tomar decisiones en las estructuras de control. Permiten evaluar condiciones y ejecutar diferentes bloques de código basados en el resultado de estas evaluaciones.

B_ Textos en Python

```
miTexto = "Python"  
print( type(miTexto) )
```



BigBayData.com | "Data is the new Bacon_" ❤️ 🖥️

```
booleana = "false";  
no = Boolean(cadena  
g(booleano);
```

Tipos de variables

Enteros (int): Representan valores numéricos enteros. Se utilizan para contar o realizar operaciones matemáticas que no requieran fracciones o decimales.

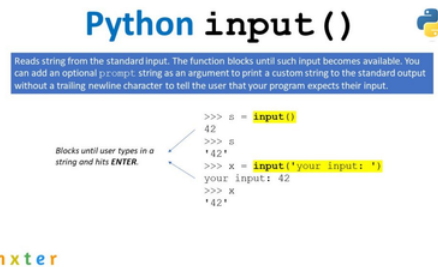
```
edad = 25  
cantidad_productos = 1
```

Flotantes (float): Representan valores numéricos que incluyen una parte decimal. Se utilizan para manejar y realizar operaciones con números que requieren una mayor precisión.

```
precio = 19.99  
pi = 3.14159
```

La función input()

La función input() en Python es fundamental para la interacción entre el programa y el usuario. Se utiliza para capturar datos ingresados por el usuario a través de la consola, permitiendo una comunicación dinámica y receptiva. Su uso adecuado mejora la experiencia del usuario y amplía las posibilidades de desarrollo de aplicaciones interactivas.



Funcionamiento

La función input() sigue estos pasos:

Mensaje de solicitud (opcional): La función puede mostrar un mensaje en pantalla para guiar al usuario sobre el tipo de información que se espera. Por ejemplo, "Ingresa su nombre: ".

Entrada del usuario: El programa se detiene y espera a que el usuario escriba un valor y presione la tecla Enter.

Captura y retorno: La función captura el texto ingresado por el usuario y lo devuelve como un valor de tipo cadena de texto (str).

En el siguiente ejemplo, el programa solicita al usuario que ingrese su nombre y luego muestra un saludo personalizado.

Las funciones int()

En Python, a menudo es necesario convertir cadenas de texto a números para realizar cálculos u otras operaciones. Las funciones `int()` y `float()` son herramientas útiles para realizar este tipo de conversiones.

Convertir a entero (int)

La función `int()` convierte una cadena de texto en un número entero. Si la cadena contiene un número entero válido, la función devuelve ese número.

```
texto_entero = "123"  
numero_entero = int(texto_entero)  
print(numero_entero)  # Salida: 123
```

Si la cadena no contiene un número entero válido, la función generará un error `ValueError`.

```
texto_no_entero = "abc"  
try:  
    numero_no_entero = int(texto_no_entero)  
except ValueError:  
    print("Error: La cadena no es un número entero válido")
```

Las funciones float()

La función float() convierte una cadena de texto en un número decimal. Si la cadena contiene un número decimal válido, la función devuelve ese número.

```
texto_flotante = "3.14"  
numero_flotante = float(texto_flotante)  
print(numero_flotante) # Salida: 3.14
```

Si la cadena no contiene un número decimal válido, la función generará un error ValueError. Hay que tener en cuenta que al realizar conversiones de texto a numérico, la cadena sea correcta y esté formateada correctamente para evitar errores.

```
texto_no_flotante = "abc"  
try:  
    numero_no_flotante = float(texto_no_flotante)  
except ValueError:  
    print("Error: La cadena no es un número decimal válido")
```

Aplicaciones

La función `input()` cuenta con aplicaciones tan variadas como:

Recopilación de datos: Se utiliza para obtener información del usuario, como nombres, números, opciones de menú u otros datos específicos para realizar cálculos o configuraciones. Por ejemplo, puede usarse en un formulario de registro para recopilar datos personales del usuario.

Validación de datos: Permite verificar que el usuario ingrese datos válidos y del tipo correcto (números, fechas, etc.). Por ejemplo, puedes utilizar `input()` para solicitar una dirección de correo electrónico y luego validar su formato.

Interfaz de usuario: Es fundamental para crear interfaces de usuario interactivas en la consola, permitiendo la comunicación entre el programa y el usuario. Un ejemplo es la creación de menús de navegación en un programa de consola.

Aplicaciones diversas: Se utiliza en diversos tipos de programas, como juegos, calculadoras, herramientas de configuración, simuladores, etc. La flexibilidad de `input()` permite su uso en muchas situaciones que requieren interacción con el usuario.

En resumen, la función `input()` en Python no se limita a la simple captura de texto. Es una herramienta versátil que permite la interacción con el usuario, la validación de datos, la creación de interfaces interactivas y la implementación de diversas funcionalidades en una amplia gama de aplicaciones.



Los objetos en Python

Objetivo

Comprender que En programación, los objetos son entidades fundamentales que representan datos y el comportamiento asociado a esos datos. A diferencia de los tipos de datos simples, como números o cadenas de texto, los objetos encapsulan tanto los datos como las operaciones que se pueden realizar sobre ellos.



Python, un lenguaje orientado a objetos

Python es un lenguaje de programación que sigue el paradigma de la programación orientada a objetos (POO). Esto significa que en Python, casi todo se implementa como un objeto o una instancia de una clase. Las clases en Python actúan como plantillas para crear objetos, definiendo sus **atributos** (características) y **métodos** (comportamientos).



En términos de POO en Python, existen tres conceptos clave que debes entender:

- **Todo es un objeto:** En Python, casi todo lo que usas es un objeto. Desde números y cadenas de texto hasta listas y funciones, todos son instancias de clases.
- **Clases como plantillas:** Una clase en Python actúa como un plano para crear objetos. Define los atributos (características) y métodos (comportamientos) que los objetos de esa clase tendrán.
- **Objetos como instancias:** Un objeto es una instancia concreta de una clase. Por ejemplo, si tienes una clase llamada "Perro", puedes crear múltiples objetos "Perro", cada uno con sus propias características (nombre, raza, color) y comportamientos (ladrar, correr).

Ventajas

La programación orientada a objetos ofrece algunas ventajas significativas:

- **Modularidad:** Permite dividir el programa en módulos independientes y reutilizables.
- **Reusabilidad:** Facilita el uso y la adaptación de código existente en nuevos proyectos.
- **Abstracción:** Oculta los detalles de implementación, permitiendo enfocarse en la lógica del programa.
- **Polimorfismo:** Permite que diferentes objetos respondan al mismo mensaje de manera diferente.
- **Herencia:** Facilita la creación de nuevas clases basadas en clases existentes, promoviendo la reutilización y la extensión del código.

La programación orientada a objetos es una forma poderosa de conceptualizar y diseñar programas. En Python, entender los conceptos de clases, objetos y herencia es fundamental para escribir código limpio, eficiente y reutilizable, lo que te permitirá desarrollar programas más complejos y sofisticados.



Tipos de objetos

Python ofrece una variedad de tipos de objetos para representar diferentes tipos de datos y estructuras. Cada tipo tiene características y usos específicos que lo hacen adecuado para distintas tareas:

- **Cadenas de texto:** Representan secuencias de caracteres alfanuméricos y símbolos. Son útiles para manipular y analizar datos textuales. Se pueden crear usando comillas simples (') o dobles (").
- **Listas:** Representan colecciones ordenadas de objetos. Pueden almacenar elementos de diferentes tipos. Son ideales para almacenar y gestionar conjuntos de datos que pueden cambiar o ser modificados. Se crean usando corchetes [].
- **Diccionarios:** Representan colecciones no ordenadas de pares clave-valor. Las claves deben ser únicas y los valores pueden ser de cualquier tipo. Son útiles para almacenar datos que se pueden acceder a través de una clave, facilitando la búsqueda y recuperación de información. Se crean usando llaves { }.
- **Tuplas:** Representan colecciones ordenadas e inmutables de elementos. No se pueden modificar una vez creadas, por lo que son adecuadas para almacenar datos que no deben cambiar a lo largo del programa. Se crean usando paréntesis ().



Operaciones básicas con cadenas de texto

Crear una cadena:

Las cadenas de texto se pueden crear utilizando comillas simples (') o dobles ("). El contenido entre las comillas representa la secuencia de caracteres que conforma la cadena.

```
cadena = "Hola, soy una cadena en Python"
```

Rebanar una cadena (slicing):

El "slicing" o rebanado de cadenas permite extraer subcadenas a partir de una cadena original. Para ello, se utiliza una notación que indica los índices de inicio y fin de la porción de la cadena que se desea extraer. El formato general es `cadena[inicio:fin]`, donde inicio es el índice del primer carácter a incluir y fin es el índice del carácter donde se detendrá el rebanado.

```
cadena = "Python"  
subcadena = cadena[1:4]    # yth
```

Operaciones básicas con cadenas de texto

Concatenar cadenas:

La concatenación de cadenas permite unir dos o más cadenas para formar una nueva cadena más larga. Esto se logra utilizando el operador +, que combina las cadenas en el orden en que se especifican.

```
nombre = "Juan"
apellido = "Pérez"
nombre_completo = nombre + " " + apellido
print(nombre_completo)  # Salida: Juan
Pérez
```

Para agregar un espacio entre las cadenas concatenadas, puedes incluir un espacio dentro de comillas, como en el ejemplo anterior (" ").

¡Recuerda! Si deseas concatenar un número con una cadena, primero debes convertir el número a una cadena usando la función `str()`.

La concatenación es una operación muy común y útil en Python, especialmente cuando se construyen mensajes o se manipulan textos de manera dinámica.

Operaciones básicas con cadenas de texto

- **Buscar subcadenas (find, index):**

En Python, los métodos `find()` e `index()` permiten buscar la primera aparición de un carácter o subcadena dentro de una cadena de texto. Ambos métodos devuelven la posición del primer carácter coincidente, pero existen diferencias clave en su comportamiento cuando la subcadena no se encuentra.

- `find`: si la cadena no se encuentra devuelve -1.
- `index`: si la subcadena no se encuentra devuelve `ValueError`.

```
cadena = "Hola, mundo"
posicion1 = cadena.find("mundo")    # 6
posicion2 = cadena.index("mundo")   # 6
```

Operaciones básicas con cadenas de texto

Comprobar si una cadena comienza o termina con cierto texto (startswith, endswith):

En Python, los métodos startswith() y endswith() son útiles para verificar si una cadena comienza o termina con un texto específico. Ambos métodos devuelven True si la condición se cumple y False en caso contrario.

```
cadena = "Hola, mundo"
comienza_con_hola = cadena.startswith("Hola")    # True
termina_con_mundo = cadena.endswith("mundo")     # True
```

Longitud de una cadena (len):

La longitud de una cadena se refiere a la cantidad de caracteres que la componen, incluidos espacios y signos de puntuación. En Python, para obtener la longitud de una cadena, se utiliza la función len(), que devuelve un número entero correspondiente al total de caracteres en la cadena.

```
cadena = "Hola, mundo"
longitud = len(cadena)
print(longitud)    # Salida: 11
```

Operaciones básicas con cadenas de texto

Acceder a caracteres por índice:

En Python, se puede acceder a un carácter individual de una cadena utilizando su índice. Los índices comienzan en 0 para el primer carácter y aumentan de uno en uno a medida que avanzas en la cadena. Además, es posible utilizar índices negativos para acceder a los caracteres desde el final de la cadena.

```
cadena = "Python"
primer_caracter = cadena[0]    # P
segundo_caracter = cadena[1]  # y
ultimo_caracter = cadena[-1]  # n
```

Métodos de cadenas:

Las cadenas de texto en Python ofrecen una amplia gama de métodos predefinidos que permiten realizar diversas operaciones sobre ellas. Estos métodos facilitan la manipulación y transformación de textos, desde cambiar su formato hasta reemplazar partes específicas de la cadena.

- **upper ()**: convierte todos los caracteres de la cadena a mayúscula
- **lower()**: convierte todos los caracteres de la cadena a minúsculas.
- **capitalize()**: convierte el primer carácter de la cadena a mayúscula y los demás a minúscula.
- **replace()**: Reemplaza todas las ocurrencias de una subcadena (old) con otra (new) en la cadena original

Operaciones básicas con cadenas de texto

Dividir una cadena (split):

El método `split()` se utiliza para dividir una cadena en una lista de subcadenas, utilizando un delimitador especificado. Si no se especifica ningún delimitador, usará por defecto cualquier espacio en blanco (espacios, tabulaciones, etc.) como delimitador.

```
cadena = "Hola,amigo,cómo,estás"
lista_palabras = cadena.split(",") #
['Hola', 'amigo', 'cómo', 'estás']
```

El **método strip** se utiliza para eliminar los espacios en blanco (u otros caracteres especificados) al inicio y al final de una cadena. No afecta a los caracteres en el medio de la cadena.

```
cadena = "  Hola, mundo  "
cadena_limpia = cadena.strip() # "Hola,
mundo"
```

Listas

Las listas en Python son estructuras de datos flexibles que permiten almacenar y organizar una colección de elementos de manera ordenada y mutable. Cada elemento en una lista está asociado a un índice que comienza en 0 y aumenta secuencialmente. A diferencia de las tuplas, las listas son mutables, lo que significa que puedes modificar, agregar o eliminar elementos después de haber creado la lista.

Ventajas

Las listas en Python destacan por su gran **versatilidad**, lo que las convierte en una herramienta fundamental para cualquier programador. Su capacidad para almacenar y organizar una amplia gama de datos las hace ideales para una gran variedad de aplicaciones, desde el procesamiento de datos simple hasta el desarrollo de aplicaciones complejas.

La sintaxis simple e intuitiva para crear, acceder y manipular listas las convierte en una herramienta **fácil de aprender y usar**, incluso para principiantes. Además, su **eficiencia** en el manejo de colecciones de datos, especialmente cuando se accede a los elementos por índice, las hace ideales para optimizar el rendimiento de los programas.



Operaciones básicas con listas

Crear lista vacía:

Para crear una lista sin elementos iniciales, se utilizan corchetes vacíos. Esto te permite iniciar con una lista vacía a la que puedes agregar elementos posteriormente

```
lista_vacia = []
```

Acceder a elementos por índice:

En una lista, puedes acceder a los elementos utilizando su índice. El primer elemento se encuentra en el índice 0, el segundo en el índice 1, y así sucesivamente. También puedes usar índices negativos para acceder a los elementos desde el final de la lista, siendo -1 el último elemento.

```
numeros = [1, 2, 3, 4, 5]
primer_elemento = numeros[0]    # 1
segundo_elemento = numeros[1]   # 2
ultimo_elemento = numeros[-1]   # 5
```

Operaciones básicas con listas

Ordenar una lista (sort):

El método sort() ordena los elementos de la lista en orden ascendente por defecto.

Si deseas ordenar en orden descendente, puedes utilizar el parámetro reverse=True.

```
numeros = [5, 3, 1, 4, 2]
numeros.sort()
print(numeros)
# Salida: [1, 2, 3, 4, 5]
```

Crear lista con elementos:

Se pueden especificar los elementos de la lista al momento de su creación, separados por comas y dentro de corchetes: lista = [elemento1, elemento2, ..., elementon].

```
numeros = [1, 2, 3, 4, 5]
```

Operaciones básicas con listas

Rebanar una lista (slicing):

El rebanado de listas te permite extraer una sublista de la lista original. La sintaxis es `lista[inicio:fin:incremento]`, donde `inicio` es el índice del primer elemento a incluir, `fin` es el índice del primer elemento a excluir (opcional), e `incremento` define el salto entre elementos (por defecto es 1).

```
numeros = [1, 2, 3, 4, 5]
```

```
sublista = numeros[1:4] # [2, 3, 4]
```

Eliminar elementos por índice (del):

El operador `del` se utiliza para eliminar un elemento de una lista especificando su índice. La eliminación modifica la lista original, removiendo el elemento en la posición indicada.

```
numeros = [1, 2, 3, 4, 5]  
del numeros[0]  
print(numeros)  
# Salida: [2, 3, 4, 5]
```

Diccionarios y tuplas

Al igual que las listas, los diccionarios y las tuplas son estructuras de datos fundamentales en Python que permiten organizar y gestionar información de manera eficiente. Los diccionarios son expertos en la organización por clave-valor, ya que almacenan pares de datos asociados entre sí facilitando el acceso a información específica. Por su parte, las tuplas guardan colecciones ordenadas de elementos que no se pueden modificar una vez creadas, brindando seguridad y confiabilidad en escenarios donde la información debe permanecer intacta.

Los diccionarios

Los diccionarios en Python son una estructura de datos que permite almacenar y organizar datos de manera asociativa, es decir, en forma de pares clave-valor. Cada elemento en un diccionario se compone de dos partes:

- Una **clave única** que actúa como identificador.
- Un **valor asociado** a esa clave.

La clave debe ser inmutable, lo que significa que puede ser una cadena, un número, una tupla u otro tipo de dato que no pueda cambiar. Los valores, en cambio, pueden ser de cualquier tipo, como números, cadenas, listas, tuplas, e incluso, otros diccionarios.

Los diccionarios se definen utilizando llaves `{ }` y los pares clave-valor se separan por dos puntos `:`. Los elementos del diccionario están separados por comas.

Diccionarios y tuplas

```
# Crear un diccionario con varios pares clave-valor
persona = {
    "nombre": "Juan",
    "edad": 30,
    "ciudad": "Madrid",
    "telefonos": ["123456789", "987654321"]
}
```

En este ejemplo, "nombre", "edad", "ciudad", y "telefonos" son las claves, y "Juan", 30, "Madrid", y ["123456789", "987654321"] son los valores asociados a esas claves, respectivamente.

Operaciones básicas con diccionarios

Crear un diccionario:

Se utiliza la notación {} con pares clave-valor separados por comas y dos puntos.



```
mi_diccionario = {"nombre": "Juan", "edad": 30, "profesion": "Ingeniero"}
```

Operaciones básicas con diccionarios

Acceder a valores por clave:

Para obtener el valor asociado a una clave específica, se utiliza la clave entre corchetes [].

```
mi_diccionario = {"nombre": "Juan", "edad": 30, "profesion": "Ingeniero"}
nombre = mi_diccionario["nombre"]
edad = mi_diccionario["edad"]
print(nombre) # Salida: "Juan"
print(edad)   # Salida: 30
```

Modificar valores en diccionario:

Para modificar el valor asociado a una clave existente, se asigna un nuevo valor a esa clave.

```
mi_diccionario = {"nombre": "Juan", "edad": 30, "profesion": "Ingeniero"}
mi_diccionario["edad"] = 31
print(mi_diccionario) # Salida: {"nombre": "Juan", "edad": 31, "profesion":
                        "Ingeniero"}
```

Operaciones básicas con diccionarios

Agregar nuevos valores al diccionario:

Para añadir un nuevo par clave-valor, se asigna un valor a una clave que aún no existe en el diccionario.

```
mi_diccionario = {"nombre": "Juan", "edad": 30}
mi_diccionario["profesion"] = "Ingeniero"
print(mi_diccionario) # Salida: {"nombre": "Juan", "edad": 30, "profesion":
"Ingeniero"}
```

Eliminar elementos del diccionario:

Para eliminar un elemento del diccionario, se utiliza la palabra clave del seguida de la clave del elemento que se desea eliminar.

```
mi_diccionario = {"nombre": "Juan", "edad": 30, "profesion": "Ingeniero"}
del mi_diccionario["edad"]
print(mi_diccionario) # Salida: {"nombre": "Juan", "profesion": "Ingeniero"}
```

Operaciones básicas con diccionarios

Verificar si una clave existe en el diccionario:

Se puede comprobar si una clave está presente en el diccionario utilizando el operador in.

```
mi_diccionario = {"nombre": "Juan", "edad": 30, "profesion": "Ingeniero"}
existe_nombre = "nombre" in mi_diccionario
existe_apellido = "apellido" in mi_diccionario
print(existe_nombre)    # Salida: True
print(existe_apellido)  # Salida: False
```



Las tuplas

En Python, las tuplas son estructuras de datos que se asemejan a las listas en cuanto a su capacidad para almacenar colecciones de elementos. Sin embargo, la principal diferencia entre tuplas y listas es que las tuplas **son inmutables**. Esto significa que, una vez creada una tupla, no puedes modificar sus elementos ni añadir nuevos elementos a ella. Esta característica de inmutabilidad hace que las tuplas sean útiles para almacenar un conjunto de valores relacionados que deben permanecer constantes durante la ejecución del programa.

Las tuplas se definen utilizando paréntesis **()** y los elementos dentro de la tupla se separan por comas.

```
# Crear una tupla con elementos variados  
mi_tupla = (1, "dos", 3.0, (4, 5))
```

Operaciones básicas con tuplas

Crear una tupla:

Se utilizan paréntesis () para crear una tupla. Los elementos se separan por comas.

```
mi_tupla = (1, 2, 3, 4, 5)
```

Acceder a elementos de una tupla por índice:

Al igual que las listas, se utiliza el índice entre corchetes para acceder a elementos específicos.

```
mi_tupla = (10, 20, 30, 40, 50)
primer_elemento = mi_tupla[0]    # 10
ultimo_elemento = mi_tupla[-1]   # 5
```



Operaciones básicas con tuplas

Rebanar una tupla (slicing):

El slicing permite extraer sub-tuplas de la tupla original. Se utiliza la sintaxis `tupla[inicio:fin:paso]` donde:

inicio: índice inicial (inclusivo) del elemento a extraer.

fin: índice final (exclusivo) del elemento a extraer.

paso: define el salto entre elementos a extraer (opcional, por defecto es 1).

```
mi_tupla = (10, 20, 30, 40, 50)
subtupla = mi_tupla[1:4]    # (20, 30, 40)
```

Longitud de una tupla (len):

La función `len` devuelve la cantidad de elementos en la tupla.

```
mi_tupla = (1, 2, 3, 4, 5)
longitud = len(mi_tupla)
print(longitud)    # Salida: 5
```

Aunque las tuplas son inmutables, **los objetos que contienen pueden ser mutables**. Por ejemplo, si una tupla contiene una lista, puedes modificar los elementos de la lista, pero no puedes reasignar una nueva lista a la tupla. Esto las hace útiles para almacenar conjuntos de datos que no deben cambiar.

Operaciones básicas con tuplas

Además, las tuplas son útiles en situaciones donde necesitas un conjunto de elementos inmutables. También son prácticas para desempaquetar múltiples valores de una función que devuelve una tupla. Por ejemplo, si una función devuelve una tupla que contiene un nombre, una edad y una ciudad, puedes desempaquetar esta tupla en tres variables separadas para acceder a cada valor de forma individual:

```
def obtener_datos():  
    return ("Juan", 30, "Madrid")  
  
nombre, edad, ciudad = obtener_datos()  
  
print(nombre)  # Salida: Juan  
print(edad)    # Salida: 30  
print(ciudad)  # Salida: Madrid
```

Operaciones básicas con listas

Ordenar una lista (sort):

El método sort() ordena los elementos de la lista en orden ascendente por defecto.

Si deseas ordenar en orden descendente, puedes utilizar el parámetro reverse=True.

```
numeros = [5, 3, 1, 4, 2]
numeros.sort()
print(numeros)
# Salida: [1, 2, 3, 4, 5]
```

Crear lista con elementos:

Se pueden especificar los elementos de la lista al momento de su creación, separados por comas y dentro de corchetes: lista = [elemento1, elemento2, ..., elementon].

```
numeros = [1, 2, 3, 4, 5]
```

Estructuras de control

Objetivo

Es comprender en la programación, las estructuras de control son herramientas esenciales para tomar decisiones y controlar el flujo de ejecución de un programa. Python ofrece una variedad de estructuras de control, incluyendo la instrucción if para la toma de decisiones condicionales, los bucles for y while para la repetición de código, y las instrucciones break y continue para controlar el flujo dentro de los bucles.



Estructuras de control

IF y ELSE

La instrucción `if` permite ejecutar diferentes bloques de código en función de una condición que se cumpla o no (verdadera o falsa), lo que se denomina expresión booleana.

La instrucción `else` se utiliza como complemento de la instrucción `if` para crear bloques de código condicional. Su función principal es ejecutar un bloque de código alternativo cuando la condición especificada en la instrucción `if` se evalúa como `False`.

FOR y WHILE

Los bucles `"for"` y `"while"` permiten repetir un bloque de código un número determinado de veces o mientras se cumpla una condición, respectivamente.

Los bucles `for` son útiles para recorrer secuencias de elementos, mientras que los bucles `while` se utilizan para repetir código hasta que se cumpla una condición específica.



Estructuras de control

BREAK y CONTINUE

Las instrucciones break y continue permiten salir de un bucle antes de que se complete su recorrido normal u omitir el resto del código en la iteración actual y saltar a la siguiente iteración, respectivamente.

La combinación de estas estructuras de control permite a los programadores crear algoritmos más complejos, tomar decisiones inteligentes y resolver problemas diversos de manera eficiente en Python. Desde simples cálculos hasta sofisticados sistemas de inteligencia artificial, las estructuras de control son los cimientos sobre los que se construyen los programas.



Estructuras de control

La **sentencia "if"** en Python se utiliza para realizar evaluaciones condicionales y ejecutar un bloque de código si la condición especificada es verdadera. La estructura general de la sentencia "if" es la siguiente:

```
if condicion:  
    # código a ejecutar si la condición es verdadera
```

Ejemplos

Verificar si un número es positivo

```
numero = float(input("Ingrese un número: "))  
if numero > 0:  
    print("El número es positivo.")
```

Estructuras de control

La sentencia if con else en Python permite gestionar dos posibles caminos de ejecución basados en una condición. Si la condición especificada en el if es verdadera, se ejecuta el bloque de código asociado a if. Si la condición es falsa, se ejecuta el bloque de código en la sección else.

La estructura general de la sentencia "if" con "else" es la siguiente:

```
if condicion:
    # código a ejecutar si la condición es verdadera
else:
    # código a ejecutar si la condición es falsa
```

Ejemplos

Verificar si un número es par o impar

```
numero = int(input("Ingrese un número entero: "))
if numero % 2 == 0:
    print("El número es par.")
else:
    print("El número es impar.")
```

Este código solicita al usuario que ingrese un número entero. Luego, utiliza la operación de módulo (%) para verificar si el número es divisible por 2 sin dejar residuo.

- Si `numero % 2 == 0` es verdadero, significa que el número es par, por lo que se imprime "El número es par."
- Si `numero % 2 != 0` es falso, el número es impar, y se imprime "El número es impar."

Estructuras de control

La sentencia "elif" (abreviatura de "else if") se utiliza para **agregar múltiples condiciones adicionales** después del "if" y antes del "else". Esto permite realizar una serie de comprobaciones condicionales en cascada. La estructura general de la sentencia "if" con "elif" y "else" es la siguiente:

```
if condicion_1:
    # código a ejecutar si la condición_1 es verdadera
elif condicion_2:
    # código a ejecutar si la condición_1 es falsa y la condición_2 es verdadera
elif condicion_3:
    # código a ejecutar si la condición_1 y condicion_2 son falsas y la
condición_3 es verdadera
...
else:
    # código a ejecutar si ninguna de las condiciones anteriores es verdadera
```

Estructuras de control

Ejemplos

Clasificación de la nota en un examen

```
nota = int(input("Ingrese la nota del examen: "))  
if nota >= 90:  
    print("Calificación: A")  
elif 80 <= nota < 90:  
    print("Calificación: B")  
elif 70 <= nota < 80:  
    print("Calificación: C")  
elif 60 <= nota < 70:  
    print("Calificación: D")  
else:  
    print("Calificación: F")
```

Estructuras de control

Ejemplos

Este programa clasifica la edad de una persona en "menor de edad", "adulto" o "adulto mayor" según unos rangos de edades establecidos.

```
edad = int(input("Ingresa la edad: "))  
if edad < 18:  
    print("Eres menor de edad.")  
elif 18 <= edad < 65:  
    print("Eres adulto.")  
else:  
    print("Eres un adulto mayor.")
```

Estructuras de control

La instrucción `pass` en Python es útil para mantener la sintaxis correcta en tu código mientras dejas un lugar para implementaciones futuras. La palabra clave "`pass`" simplemente indica que no se hará nada en ese bloque de código y se pasa a la siguiente instrucción.

Usos

La instrucción `pass` en Python es una declaración nula que no hace nada. Se usa principalmente en los siguientes casos:

- **Desarrollo de código:** Cuando estás escribiendo código y quieres definir una estructura, como una función, clase o bucle, pero aún no has implementado el contenido. Usar `pass` permite que el código pase sintácticamente sin errores.
- **Estructuras incompletas:** Para crear estructuras que aún no están completas pero que deben estar presentes para completar la sintaxis del lenguaje.
- **Espacios para implementaciones futuras:** Como marcador de posición para bloques de código que agregarás más adelante.



Estructuras de control

Ejemplos

Usando "pass" en un bloque de código

```
x = 10
if x > 5:
    # Bloque vacío para manejar el caso en que x es mayor que 5
    pass
else:
    print("x es menor o igual a 5.")
```

Estructuras de control

El bucle for

En Python, los bucles "for" se utilizan para **iterar sobre una secuencia** (como listas, tuplas, diccionarios, cadenas, etc.) y realizar una **acción en cada elemento** de la secuencia. La estructura general de un bucle "for" es la siguiente:

Donde:

- **"elemento"** es una variable que tomará el valor de cada elemento de la secuencia en cada iteración del bucle.
- **"secuencia"** es la colección de elementos sobre la cual se iterará, por ejemplo, una lista o una cadena.

```
for elemento in secuencia:  
    # código a ejecutar en cada iteración
```

Estructuras de control

Ejemplos

Iterar sobre un rango específico, La función `range(1, 6)` genera una secuencia de números enteros desde 1 hasta 5 (el límite superior 6 no se incluye). El bucle `for` recorre esta secuencia e imprime cada número.

```
for numero in range(1, 6):  
    print(numero)
```

La función range

La función range es una función incorporada en Python que genera una **secuencia de números dentro de un rango** especificado, y es ampliamente utilizada en bucles for para iterar un número determinado de veces.

```
range(start, stop, step)
```

Ejemplos

```
for num in range(10):  
    print(num)
```

Donde:

- **start:** Es el valor inicial de la secuencia. Es opcional; si no se proporciona, el valor predeterminado es 0.
- **stop:** Es el valor final de la secuencia. El rango se genera hasta este valor, pero sin incluirlo. Es obligatorio.
- **step:** Es el incremento entre los valores. Es opcional; el valor predeterminado es 1.

Generar una secuencia de números pares del 0 al 20

La función `range(0, 21, 2)` genera números del 0 al 20 con un incremento de 2, produciendo una secuencia de números pares. El bucle `for` imprime cada número en esa secuencia.

```
for num in range(0, 21, 2):  
    print(num)
```

Contar vocales en una cadena

El bucle for recorre cada letra en la cadena texto (convertida a minúsculas) y cuenta cuántas veces aparece una vocal. El contador contador_vocales almacena el número total de vocales.

Crear una lista usando "range()" casa hola como estas casa

```
texto = "Hola, cómo estás?"
vocales = "aeiou"
contador_vocales = 0
for letra in texto.lower():
    if letra in vocales:
        contador_vocales += 1
print("El número de vocales en el texto es:", contador_vocales)
```

Crear una lista usando "range()"

La función `range(1, 6)` genera una secuencia de números del 1 al 5. El bucle `for` agrega cada número a la lista `numeros`. Finalmente, se imprime la lista completa.

```
numeros = []  
  
for i in range(1, 6):  
    numeros.append(i)  
  
print(numeros)
```

Iterar sobre los valores de un diccionario

El bucle for recorre todos los valores del diccionario alumno utilizando el método values(), e imprime cada valor.

```
alumno = {  
    "nombre": "Juan",  
    "edad": 25,  
    "nota": 8.5  
}  
  
for valor in alumno.values():  
    print(valor)
```

Iterar sobre las claves y valores de un diccionario

El bucle for recorre cada par clave-valor en el diccionario alumno utilizando el método items(), e imprime cada clave junto con su valor asociado.

```
alumno = {  
    "nombre": "Juan",  
    "edad": 25,  
    "nota": 8.5  
}  
  
for clave, valor in alumno.items():  
    print(f"{clave}: {valor}")
```

Estructuras de control

El bucle while

El bucle while es una estructura de control que permite ejecutar repetidamente un bloque de código mientras una condición específica sea verdadera. Es útil para situaciones donde el número de iteraciones no se conoce de antemano, y depende de que se cumpla o deje de cumplirse una condición.

```
while condicion:  
    # código a ejecutar mientras la condición sea verdadera
```

Donde:

- "condición" es una expresión que se evalúa antes de cada iteración del bucle. Si la condición es verdadera (True), se ejecuta el bloque de código dentro del bucle. Si es falsa (False), el bucle se detiene y el programa continúa con la siguiente línea de código fuera del bucle.

Ejemplos

Contador regresivo

```
contador = 5  
while contador > 0:  
    print(contador)  
    contador -= 1
```

Estructuras de control

Los comandos break y continue

Los comandos "break" y "continue" son dos instrucciones que se utilizan dentro de los bucles "while" en Python para controlar el flujo de ejecución. A continuación, te explicaré ambos con ejemplos.

- **Comando "break"**: Se utiliza para detener la ejecución del bucle de manera prematura cuando se cumple cierta condición. Esto significa que, cuando se encuentre la instrucción "break" dentro del bucle, la ejecución saldrá del bucle y continuará con la siguiente instrucción.
- **Comando "continue"**: Se utiliza para pasar a la siguiente iteración del bucle de manera inmediata cuando se cumple una cierta condición. Si se encuentra la instrucción "continue" dentro del bucle, el resto del bloque de código en esa iteración se ignorará, y la ejecución volverá a la parte superior del bucle para continuar con la siguiente iteración.



Búsqueda de un número en una lista

En este ejemplo, el programa busca un número ingresado por el usuario dentro de la lista "numeros". Si encuentra el número, imprimirá "Número encontrado en la lista." y se detendrá usando la instrucción "break". Si recorre toda la lista sin encontrar el número, imprimirá "Número no encontrado en la lista." y también se detendrá con "break".

```
numeros = [10, 25, 30, 45, 50]
numero_buscado = int(input("Ingrese un número a buscar: "))
encontrado = False

while not encontrado:
    for numero in numeros:
        if numero == numero_buscado:
            print("Número encontrado en la lista.")
            encontrado = True
            break
    else:
        print("Número no encontrado en la lista.")
        break
```

Imprimir sólo números pares

En este ejemplo, el programa utiliza un bucle "while" para imprimir los números pares del 1 al 10. Si el número actual es impar (es decir, su residuo al dividir por 2 es diferente de cero), se utilizará "continue" para pasar a la siguiente iteración sin imprimir el número impar. Si el número es par, se imprimirá y luego se aumentará el valor del número en 1 para continuar con la siguiente iteración.

```
numero = 1
while numero <= 10:
    if numero % 2 != 0:
        numero += 1
        continue
    print(numero)
    numero += 1
```

Estructuras de control

Los comandos **break** y **continue**

Ambos comandos permiten un control flexible y eficiente del flujo de bucles, lo que los hace esenciales para escribir programas más dinámicos y adaptables.

- **break:** Es ideal para salir de un bucle antes de que haya terminado todas las iteraciones previstas, útil cuando se ha alcanzado un resultado satisfactorio o cuando se detecta una condición de salida.
- **continue:** Permite omitir el resto del código en una iteración específica y pasar directamente a la siguiente, útil cuando quieres evitar la ejecución de código bajo ciertas condiciones, pero continuar iterando.



Funciones

Objetivo

Es comprender en la programación, En Python, las funciones son bloques de código reutilizables que realizan una tarea específica. Definir funciones te permite organizar y estructurar mejor tu código, así como evitar la repetición. Aquí te explico con más detalle el proceso de creación de una función y su uso



Funciones

Puedes definir tus propias funciones utilizando la **palabra clave def, seguida del nombre de la función y los parámetros** que aceptará. Luego, el bloque de código de la función debe ir indentado

```
def nombre_de_la_funcion(parametros):  
    # Código que realiza alguna tarea  
    return resultado # Opcional, si la f
```

Ejemplos

Función para sumar dos números

Aquí tienes un ejemplo simple de cómo crear una función que suma dos números:

```
def suma(a, b):  
    resultado = a + b  
    return resultado
```

En este ejemplo, hemos definido una función llamada suma que toma dos parámetros a y b. Dentro de la función, sumamos los dos números y almacenamos el resultado en una variable llamada resultado. Luego, utilizamos la declaración return para devolver el valor de resultado como resultado de la función.

Funciones

Las funciones random

La librería estándar random de Python proporciona un conjunto de funciones que permiten generar números aleatorios, así como realizar otras operaciones aleatorias, como elegir elementos de una lista, mezclar elementos de una secuencia, etc. Estas funciones son muy útiles en una variedad de aplicaciones, como simulaciones, juegos, pruebas y otros algoritmos que requieren aleatoriedad.

```
import random
```

Función randint

La función randint() devuelve un número entero aleatorio entre los valores indicados, incluyendo ambos límites. Esto es útil cuando necesitas seleccionar un elemento de una lista utilizando índices aleatorios.

```
import random

regalos = ['sartén', 'jamón', 'mp4', 'muñeca', 'tv', 'patín', 'balón', 'reloj',
'bicicleta', 'anillo']

for sorteo in range(5):
    regalo = regalos[random.randint(0, 9)]
    print('Sorteo', sorteo + 1, ':', regalo)
```

Funciones

Función choice

La función choice() selecciona un elemento al azar de una lista o secuencia.

```
transporte = ['bici', 'moto', 'coche', 'tren', 'avión']  
print('Hoy viajaré en:', random.choice(transporte))
```

Ejemplos

Función uniform

La función uniform() devuelve un número flotante aleatorio entre dos valores especificados, incluyendo ambos extremos.

```
for numero in range(3):  
    print(random.uniform(100, 105), end=' ')
```

Este código genera y muestra tres números flotantes aleatorios entre 100 y 105. La función uniform() es útil cuando necesitas un número aleatorio dentro de un rango específico que no está limitado a enteros.

¡Sigamos trabajando!